Analiza Dużych Zbiorów Danych

Wykład 2: Architektury systemów Big Data i taksonomia narzędzi

dr inż. Krzysztof Ropiak
Uniwersytet Warmińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Plan wykładu

- **1. Ewolucja paradygmatów i klasyczne wzorce architektoniczne**
  Tradycyjny ETL vs nowoczesny ELT • Presja czasu rzeczywistego • Architektura Lambda i problem podwójnego kodu • Architektura Kappa
- **2. Realia produkcyjne: Studium przypadku i Data Swamp**
  Studium wdrożeniowe FinTech "PayFast" • Jak obietnica jeziora danych skończyła się bagnem • Brak ACID na plikach • Silosy analityczne
- **3. Nowoczesny Lakehouse: Architektura Medallion i 4 warstwy potoku**
  Złoty standard Medallion (Bronze, Silver, Gold) • 4 warstwy funkcjonalne: Ingestion, Storage, Compute, Serving • Przegląd ekosystemu narzędzi
- **4. Synteza inżynierska i analiza kompromisów (Trade-offs)**
  Macierz decyzyjna Lambda vs Kappa vs Medallion • Studia decyzyjne dla sali • Podsumowanie

1. Klasyczny paradygmat hurtowni danych: Tradycyjny ETL

W tradycyjnym świecie Business Intelligence (lata 1990–2010) rządził **ETL** (*Extract, Transform, Load*):

1. Extract (Ekstrakcja)
Systemy OLTP (ERP, CRM, SQL)
- Cykliczne zapytania SQL
- Zrzuty tabel do plików CSV
- Ryzyko obciążenia transakcji

➔

2. Transform (Transformacja)
Serwer ETL (Informatica / SSIS / Talend)
- Filtrowanie i czyszczenie
- Wyliczanie agregacji w locie
- **Wąskie gardło pamięci RAM i CPU**

➔

3. Load (Ładowanie)
Hurtownia (Oracle / Teradata / DB2)
- **Schema-on-Write**
- Tylko zunifikowane tabele
- Bardzo wysoki koszt per TB

- **Drogie miejsce składowania:** Hurtownie danych kosztowały dziesiątki tysięcy dolarów za terabajt; ładowano wyłącznie dane przefiltrowane i zagregowane.
- **Schema-on-Write:** Struktura tabeli docelowej musiała być zdefiniowana przed załadowaniem. Błędny rekord zatrzymywał cały proces ładowania.
- **Serwer ETL jako wąskie gardło:** Obliczenia wykonywał pojedynczy serwer pośredniczący, który musiał pomieścić transformacje w pamięci RAM.

Źródło: Ralph Kimball, Margy Ross, "The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling", Wiley 2013.

2. Przewrót chmurowy: Przejście od ETL do nowoczesnego ELT

Dojrzałość pamięci obiektowej (S3/ADLS) oraz chmurowych silników MPP wywróciła tę kolejność:

1. Extract & Load (EL)

Konektory (Fivetran / Airbyte)

- Pobieranie 1:1 z baz i API
- Brak przekształceń w locie
- Zero logiki biznesowej

➔

2. Raw Data Storage

Object Storage (S3 / Snowflake / BigQuery)

- Tani, masowy storage w chmurze
- **Schema-on-Read**
- Niezmienna historia 1:1

➔

3. Transform wewnątrz (T)

Silnik chmurowy / dbt / Spark SQL

- Przetwarzanie rozproszone MPP
- Wersjonowany kod SQL w Git
- Elastyczne modele pod analitykę BI

- **Tani storage zmienia wszystko:** Zapisujemy *wszystkie* surowe dane bez wcześniejszego filtrowania (brak utraty kontekstu historycznego).
- **Schema-on-Read:** Schemat narzucany jest dopiero w momencie odpytywania danych, a nie podczas ich wgrywania do magazynu.
- **Silnik analityczny transformuje dane:** Zamiast zewnętrznego serwera ETL, to potężny, skalowalny silnik rozproszony wykonuje transformacje wewnątrz chmury.

Źródło: Tristan Handy, "The Modern Data Stack: Past, Present, and Future", dbt Labs Blog, 2020.

3. Inżynierskie porównanie: ETL czy ELT?

Wymiar	Tradycyjny ETL	Nowoczesny ELT
Gdzie zachodzą obliczenia?	Zewnętrzny serwer ETL (przed załadowaniem)	Docelowy silnik analityczny (po załadowaniu)
Podejście do schematu	*Schema-on-Write* (sztywny, z góry zdefiniowany)	*Schema-on-Read* (elastyczny, ewoluujący)
Retencja danych surowych	Odrzucane lub archiwizowane na taśmach	Trwale zachowane w chmurze 1:1
Czas wdrożenia nowej metryki	Tygodnie/miesiące (zmiana całego potoku ETL)	Godziny (nowe zapytanie SQL/dbt na surowych danych)
Skalowalność obliczeń	Ograniczona mocą serwera ETL	Praktycznie nieograniczona (skalowanie chmurowe)
Prywatność i RODO / GDPR	Łatwiejsza anonimizacja *w locie* przed zapisem	Wymaga rygorystycznej kontroli dostępu i masek w storage

Pułapka ELT: Przechowywanie surowych danych oznacza, że nieświadomie możemy załadować numery PESEL, karty płatnicze lub hasła do data lake. Wymaga to natychmiastowego wdrożenia automatycznego maskowania PII!

4. Presja czasu: Granice klasycznego przetwarzania wsadowego

Przez dekady analityka biznesowa opierała się na tzw. **nocnym oknie wsadowym (Nightly Batch Window)**:

Dzień (08:00 – 22:00)
Ruch transakcyjny
Transakcje klientów, zamówienia w sklepie, kliknięcia w aplikacji. Baza OLTP pod ciągłym ruchem.

➔

Noc (02:00 – 06:00)
Okno wsadowe (Batch Window)
Hadoop MapReduce / Cron ETL mielą petabajty. Jeśli job padnie o 04:30 – katastrofa.

➔

Rano (07:00)
Raport Zarządczy
Statyczny dashboard w BI z liczbami ze wczoraj. Decyzje podejmowane z opóźnieniem 24h.

Dlaczego to przestało wystarczać?

1. **Wzrost wolumenu wydłużył okno:** Przetwarzanie wsadowe, które kiedyś trwało 2 godziny, zaczęło trwać 9 godzin i "wylewać się" na godziny pracy firmy.
2. **Globalizacja biznesu (24/7):** Sklepy internetowe i platformy SaaS nie mają "nocy" – transakcje spływają bez przerwy z różnych stref czasowych.
3. **Decyzje wymagające milisekund:**
   - Wykrywanie oszustw bankowych (*Fraud Detection*) – transakcję trzeba zablokować zanim klient odejdzie od terminala.
   - Dynamiczny cennik przejazdów / biletów (*Surge Pricing*).

5. Architektura Lambda: Połączenie dwóch światów

W 2011 roku **Nathan Marz** (twórca Apache Storm) sformułował wzorzec **Architektury Lambda**:

Wszystkie Zdarzenia

Niezmienny strumień

Każde nowe zdarzenie trafia jednocześnie do dwóch torów

WARSTWA WSADOWA (Batch Layer) DOKŁADNA • WYSOKA LATENCJA

HDFS / S3 + Spark ➔ **Widoki Wsadowe** (niezmienna prawda historyczna)

WARSTWA SZYBKA (Speed Layer) MILISEKUNDY • PRZYBLIŻONA

Apache Storm / Flink ➔ **Widoki w Czasie Rzeczywistym** (ostatnie godziny)

Warstwa Serwująca

HBase / Cassandra

Zespolenie wyników:

`BatchView ⊕ RealtimeView`

> **Główna idea matematyczna:**
>
> $$Query = f(Wszystkie_Dane) \approx f(Widok_Batch) \oplus f(Widok_Realtime)$$

Źródło: Nathan Marz, James Warren, "Big Data: Principles and best practices of scalable realtime data systems", Manning 2015.

6. Klątwa Architektury Lambda: Problem podwójnego kodu

Choć Architektura Lambda wyglądała znakomicie na papierze, stała się **koszmarem inżynierów oprogramowania**:

Problem podwójnej bazy kodu (Dual Codebase Problem):
Tę samą regułę biznesową (np. wyliczanie sesji użytkownika lub obrotu) trzeba było napisać i utrzymywać **dwukrotnie** w dwóch zupełnie różnych technologiach!

1. **Różne API i języki:** Warstwa Batch w Java/MapReduce lub Scali (Spark), warstwa Speed w Clojure/Storm lub C++.
2. **Zjawisko dryfu danych (Data Drift):**
   - Drobne różnice w obsłudze stref czasowych lub zaokrągleń zmiennoprzecinkowych powodowały, że o 23:59 (widok Speed) i o 02:00 (widok Batch) raport pokazywał **inne liczby**.
3. **Piekielnie trudne testy i debugowanie:** Znalezienie błędu wymagało jednoczesnego debugowania klastra Hadoop i klastra Storm.

Źródło: Jay Kreps, "Questioning the Lambda Architecture", O'Reilly Radar, 2014.

7. Architektura Kappa: Wszystko jest strumieniem

W odpowiedzi na udrękę Lambdy, **Jay Kreps** (współtwórca Apache Kafka) zaproponował **Architekturę Kappa**:

ROZPROSZONY NIEZMIENNY LOG (Apache Kafka / Pulsar)
Pojedyncze źródło prawdy dla całego systemu (długa retencja zdarzeń)

↓

Job Strumieniowy v1 (Produkcja)
Apache Flink / Spark Streaming
Zasila tabelę produkcyjną v1 w czasie rzeczywistym.

Tabela Serwująca v1

Job Strumieniowy v2 (Reprocessing)
Nowa wersja logiki biznesowej
Czyta log od offsetu = 0, przeliczając całą historię.

Tabela Serwująca v2 (przełącz ruch po dogonieniu!)

- **Jedna baza kodu:** Wyłącznie silnik przetwarzania strumieniowego.
- **Przetwarzanie wsadowe to tylko podzbiór strumienia:** Wsad to po prostu strumień z określonym początkiem i końcem w przeszłości.

Źródło: J. Kreps, "Questioning the Lambda Architecture", O'Reilly Radar, 2014.

8. Granice Architektury Kappa: Kiedy eliminacja batcha boli?

Czy Architektura Kappa rozwiązała wszystkie problemy? **Nie.** W realiach enterprise napotkała bariery:

1. **Koszt długoterminowej retencji w logu:**
   - Apache Kafka wymaga szybkich dysków SSD/NVMe przyłączonych do brokerów. Przechowywanie w niej 5 petabajtów danych z ostatnich 7 lat jest ekonomicznym absurdem w porównaniu do chmurowego S3.
2. **Skomplikowane złączenia historyczne (Enrichment Joins):**
   - Dołączenie do bieżącego zdarzenia tabeli wymiarowej z 500 milionami klientów wymaga utrzymywania gigantycznego stanu w pamięci silnika strumieniowego (tzw. *Stateful Stream Processing*).
3. **Dane spóźnione (Late / Out-of-Order Data):**
   - Gdy zdarzenia z telefonów komórkowych docierają z opóźnieniem 3 dni (np. po wyjściu z trybu samolotowego), obsługa okien czasowych w strumieniu staje się niezwykle skomplikowana.

> **Wniosek inżynierski:** Kappa zrewolucjonizowała przetwarzanie zdarzeń, ale analityka ad-hoc na danych historycznych nadal wymagała taniego, masowego składowania plików.

9. Oś ewolucji wzorców architektonicznych

1995–2010

Tradycyjny ETL

Hurtownia danych, sztywne schematy, drogi dysk, brak czasu rzeczywistego, skalowanie pionowe.

2011–2014

Architektura Lambda

Równoległy Batch i Speed Layer. **Wada:** podwójna baza kodu, zjawisko dryftu danych.

2014–2019

Architektura Kappa

Wszystko jest logiem w Kafce. Jeden silnik, ale trudne wieloletnie złączenia i retencja.

2020+

Medallion / Lakehouse

Unifikacja wsadu i strumienia. ACID na tanim storage S3. Warstwy Bronze-Silver-Gold.

Kluczowa lekcja z dwóch dekad:

> **Architektura nie może zmuszać programistów do pisania tej samej logiki biznesowej wielokrotnie, ani wymagać kompromisu pomiędzy spójnością danych a ich świeżością.**

10. Studium przypadku: Wzrost i kryzys w FinTech "PayFast"

Wyobraźmy sobie historię dynamicznego FinTechu **"PayFast"** (studium oparte na faktach):

Faza 1: Monolit PostgreSQL

Skrajne obciążenie bazy operacyjnej

Raporty analityczne odpalane wprost na bazie produkcyjnej blokowały tabele. Generowanie bilansu kwartalnego wieszało aplikację mobilną klientów!

Faza 2: Entuzjazm „Data Lake”

Zrzut wszystkiego do AWS S3 bez nadzoru

Decyzja: *„Wyrzucamy ograniczenia! Każdy zespół zrzuca JSON-y do S3, a analitycy sami sobie to sparsują”*. Brak schematu, brak kontroli wersji.

Rezultat po 18 miesiącach:

Do S3 trafiło 800 TB danych w ponad 40 milionach plików. Koszty chmury wzrosły o 350%, a dyrektor finansowy oznajmił: *„Nie potrafimy wygenerować spójnego bilansu kwartalnego!”*

11. Anatomia katastrofy: Jak powstało "Bagno Danych" (Data Swamp)?

Obietnica: Data Lake

WIZJA

- Jedno centralne repozytorium na całą wiedzę firmy
- Brak sztywnych ograniczeń dla inżynierów
- Łatwy dostęp do danych dla analityków i Data Science
- Niskie koszty składowania w chmurze S3/ADLS

Rzeczywistość: Data Swamp

KRYZYS

- 10 000 katalogów z plikami o nieznanej strukturze
- Cicha zmiana schematów niszcząca nocne pipeline'y
- 15 wersji tego samego pliku (`users_final_v3.csv`)
- Totalny brak zaufania do danych w zarządzie

Co doprowadziło do załamania platformy PayFast?

1. **Brak kontroli schematu (*Schema Drift*):** Zmiana `client_id` na `customer_uuid` bez ostrzeżenia wywaliła 12 pipeline'ów.
2. **Katalogowy śmietnik:** Brak konwencji nazewnictwa i wersjonowania danych.
3. **Brak metadanych:** Nikt nie wiedział, kto jest właścicielem danego zbioru ani czy dane są świeże.

12. Brak transakcyjności ACID na plikach: Pół-zapisane partycje

Najgroźniejszy problem techniczny w Data Lake wynikał z samej natury pamięci obiektowej (S3/GCS):

Zapis partycji `date=2024-03-15` w surowym S3 (planowane 100 plików Parquet):

PLIKI 001–047: ZAPISANE OK ➔ PLIK 048: AWARIA KLASTRA (OOM) 💥 ➔ Brak atomowości: 47 plików zostało na dysku!

Konsekwencje braku transakcji:

- **Brak atomowości (A w ACID):** S3 nie zna pojęcia transakcji dyskowej. Zapis nie wycofał się automatycznie (*rollback*).
- **Brudny odczyt (Dirty Read):** Dashboard pobrał dokładnie 47% transakcji, wykazując zafałszowane przychody spółki!
- **Wyścigi współbieżności (Race Conditions):** Dwa równoległe procesy dopisujące do tego samego katalogu nadpisywały wzajemnie swoje metadane.

Źródło: Michael Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores", VLDB 2020.

13. Rozłam w organizacji: Zjawisko silosów analitycznych

W wyniku chaosu w Data Lake w firmie PayFast doszło do zjawiska **Silosów Danych (Data Silos)**:

Surowy Data Lake w S3

Totalny chaos plikowy

Brak gwarancji schematu i transakcyjności

Zespół BI & Raportowania (Ucieczka do Snowflake)

Kopiują dane własnymi skryptami. Raportują zarządowi: **1 200 000 aktywnych klientów**.

Zespół Data Science & ML (Praca bezpośrednio na S3)

Trenują modele na plikach S3. Ich model wykazuje: **1 480 000 klientów**.

Spotkanie Zarządu spółki pod koniec kwartału:

Dyrektor Finansowy: *„Mamy 1 200 000 aktywnych klientów."* | Dyrektor AI: *„Nasz algorytm widzi 1 480 000 klientów."*

Zarząd: **Która liczba jest prawdziwa? Kto za to odpowiada?**

14. Nowoczesne rozwiązanie: Wprowadzenie do Architektury Medallion

Odpowiedzią na bagno danych i silosy stał się wzorzec **Architektury Medallion** (zwany też *Multi-Hop Architecture*), stanowiący fundament nowoczesnego **Lakehouse**:

BRĄZOWA (Bronze)
Surowa prawda o źródłach
- Zapis 1:1, Append-Only
- Format surowy (JSON/Parquet)
- Historia i pełny audyt

➔

SREBRNA (Silver)
Oczyszczona prawda inżynierska
- Egzekwowanie typów i schematu
- Deduplikacja zdarzeń
- Pojedyncze źródło prawdy (SSOT)

➔

ZŁOTA (Gold)
Prawda biznesowa i BI
- Model Gwiazdy (Star Schema)
- Preagregowane metryki KPI
- Odpowiedź w milisekundach

- **Jeden wspólny magazyn (Lakehouse):** Łączy niski koszt obiektowego storage'u (S3/ADLS) z gwarancjami transakcyjnymi ACID.
- **Stopniowa rafinacja:** Dane przechodzą logiczne etapy czyszczenia i wzbogacania, a każda warstwa ma precyzyjnie określony cel i grupę odbiorców.

Źródło: Databricks Architecture Center, "What is the Medallion Lakehouse Architecture?", 2021.

15. Warstwa Brązowa (Bronze / Raw): Niemutowalna historia

Warstwa Brązowa to **cyfrowa czarna skrzynka** Twojej platformy danych.

Zasady inżynierskie:

- **Zapis 1:1 ze źródła:** Żadnych przekształceń biznesowych, żadnego usuwania kolumn. Dane wpadają dokładnie w takim formacie, w jakim zostały wygenerowane.
- **Tryb Append-Only (Tylko dopisywanie):** Nowe dane są wyłącznie dopisywane. Nic nie jest mutowane ani usuwane.
- **Wzbogacenie o metadane techniczne:** Każdy rekord otrzymuje techniczne kolumny audytowe:
  - `_ingested_at` (znacznik czasu pobrania),
  - `_source_file_name` lub `_source_topic` (źródło pochodzenia).

> **Dlaczego to kluczowe?**
> Jeśli za pół roku odkryjesz błąd w algorytmie przeliczania podatków w warstwie wyższej, możesz **zresetować wyższe warstwy i przeliczyć wszystko od zera**, ponieważ surowa prawda historyczna w Bronze nigdy nie została zniszczona.

16. Warstwa Srebrna (Silver): Pojedyncze źródło prawdy (SSOT)

W warstwie Srebrnej dane przekształcają się w **zaufaną tabelę inżynierską**.

Co dzieje się pomiędzy Bronze a Silver?

1. **Egzekwowanie typów i schematu (*Schema Enforcement*):** Rzutowanie ciągów znaków na daty, liczby, wartości boolowskie. Odrzucenie rekordów niespełniających kontraktu do kolejki błędów (*Dead Letter Queue*).
2. **Deduplikacja:** Usunięcie duplikatów zdarzeń wygenerowanych przez ponowienia sieciowe (*At-Least-Once Delivery*).
3. **Standaryzacja:** Zunifikowanie formatów (np. waluty przeliczone do bazowej, formaty kodów krajów ISO-3166).
4. **Wzbogacenie relacyjne (*Conforming*):** Złączenie transakcji ze słownikami klientów i produktów za pomocą transakcyjnych operacji `MERGE` (Upsert).

Docelowy odbiorca: Analitycy danych, inżynierowie eksploracji, algorytmy ML Feature Store.

17. Warstwa Złota (Gold): Zoptymalizowana pod biznes i BI

Warstwa Złota to dane gotowe do **bezpośredniej konsumpcji zarządczej i produkcyjnej**.

Warstwa Silver
10 miliardów transakcji
Znormalizowane, szczegółowe fakty transakcyjne

➔

Warstwa Gold (Agregacja)
Tabela preagregowana (~100k wierszy)
Sumy kroczące, Churn, EBITDA, Star Schema

➔

Konsumenci BI
Dashboardy PowerBI / Tableau
Odpowiedź w **50 ms** zamiast skanowania terabajtów

- **Modelowanie wymiarowe (Kimball Star Schema):** Tabele faktów otoczone tabelami wymiarów (`dim_customers` , `dim_stores` , `dim_calendar`).
- **Preagregacje biznesowe:** Wyliczone sumy kroczące, wskaźniki retencji (Churn), zysku (EBITDA), koszyka zakupowego (AOV).
- **Maksymalna wydajność odczytu:** Zapis w wysoce skompresowanych formatach kolumnowych pod zapytania analityczne.

18. Pełny schemat Architektury Medallion w praktyce

1. Źródła

Ingestion

- Bazy OLTP (CDC)
- Eventy (Kafka)
- API SaaS (Airbyte)

2. BRONZE

Raw Data

- Zapis 1:1 Append
- Parquet / Delta
- Metadane audytu

3. SILVER

Cleaned SSOT

- Deduplikacja
- Transakcyjny MERGE
- Spójne typy i relacje

4. GOLD

Business Aggregates

- PowerBI / Tableau
- Modele ML / AI
- Reverse ETL do CRM

Zasada unifikacji strumienia i wsadu: Zarówno strumień zdarzeń z Kafki, jak i nocny zrzut z bazy OLTP trafiają do warstwy **Bronze** w tym samym formacie tabelarycznym (Delta/Iceberg). Dalsza transformacja do Silver i Gold może działać w trybie wsadowym lub ciągłym mikrostrumieniowym!

19. Dekonstrukcja platformy: Cztery warstwy funkcjonalne

Aby dobrać właściwe narzędzia, musimy rozbić architekturę na **4 niezależne warstwy**:

4. WARSTWA SERWOWANIA I GOVERNANCE	Bazy OLAP, Semantyka BI, Kontrola Jakości i Katalog Metadanych
3. WARSTWA OBLICZENIOWA (COMPUTE)	Silniki rozproszone MPP, transformacje SQL (Spark, Trino, dbt)
2. WARSTWA SKŁADOWANIA I FORMATÓW	Chmurowy Object Storage (S3/ADLS) + Otwarte Formaty Tabelaryczne (Delta/Iceberg)
1. WARSTWA POZYSKIWANIA (INGESTION)	Konektory CDC (Debezium), logi strumieniowe (Kafka), narzędzia SaaS ELT

Zasada modułowości:

W nowoczesnej architekturze każdą z tych warstw można wymienić niezależnie od pozostałych, unikając uzależnienia od jednego dostawcy (*Vendor Lock-in*).

Źródło: Matt Bornstein, Martin Casado, "Emerging Architectures for Modern Data Infrastructure", Andreessen Horowitz (a16z), 2020.

20. Warstwa 1: Pozyskiwanie danych (Data Ingestion)

Jak wprowadzić dane do ekosystemu Big Data bez obciążania systemów produkcyjnych?

Trzy główne mechanizmy inżynierskie:

1. **Change Data Capture (CDC):**
   - Zamiast odpytywać bazę produkcyjną ciężkim `SELECT * FROM orders WHERE updated_at > ...`, silnik CDC czyta bezpośrednio **fizyczny log transakcyjny** bazy (Postgres WAL, MySQL Binlog).
   - Narzędzia: **Debezium**, **AWS DMS**, **Qlik Replicate**.
2. **Batch / SaaS Ingestion (Konektory gotowe):**
   - Pobieranie danych z setek API zewnętrznych (Salesforce, Stripe, Google Ads) przez znormalizowane konektory.
   - Narzędzia: **Airbyte** (Open Source), **Fivetran** (SaaS).
3. **Strumienie zdarzeń o wysokiej przepustowości (Streaming Ingestion):**
   - Przyjmowanie milionów zdarzeń telemetrycznych na sekundę do bufora logowego.
   - Narzędzia: **Apache Kafka**, **Apache Pulsar**, **AWS Kinesis**, **Azure Event Hubs**.

21. Warstwa 2: Składowanie i otwarte formaty tabelaryczne

Gdzie i jak fizycznie spoczywają dane? Podwójny poziom składowania:

A. Fizyczny nośnik obiektowy (Cloud Object Storage)

- **AWS S3**, **Azure Data Lake Storage Gen2 (ADLS)**, **Google Cloud Storage (GCS)**, **MinIO** (on-premise / S3-compatible).
- Niezwykle tani, nieograniczona pojemność, trwałość 11 dziewiątek (99.999999999%).

B. Otwarte formaty tabelaryczne (Open Table Formats)

Surowy plik Parquet to za mało. Potrzebujemy **warstwy tabelarycznej dającej gwarancje ACID**:

- **Delta Lake** (Linux Foundation / Databricks): Dziennik transakcji ACID `_delta_log/`, operacje Time Travel, optymalizacja Z-Order.
- **Apache Iceberg** (tworzony pierwotnie przez Netflix): Projektowany pod ogromną skalę, separacja metadanych od partycjonowania fizycznego.
- **Apache Hudi** (tworzony przez Uber): Zoptymalizowany pod bardzo szybkie mutacje i operacje UPSERT w czasie bliskim rzeczywistemu.

22. Warstwa 3: Silniki obliczeniowe i transformacje

Kto wykonuje ciężką pracę matematyczną i analityczną?

1. Rozproszone silniki przetwarzania ogólnego przeznaczenia

- **Apache Spark:** Bezsprzeczny król rozproszonych obliczeń wsadowych i mikrostrumieniowych w pamięci RAM.
- **Apache Flink:** Przetwarzanie strumieniowe zdarzenie po zdarzeniu (*event-by-event*) o ultraniskiej latencji.
- **Ray:** Nowoczesny silnik obliczeń rozproszonych, zoptymalizowany pod trenowanie modeli AI/ML.

2. Silniki zapytań SQL (Distributed SQL Query Engines)

- **Trino (dawniej PrestoSQL):** Silnik federacyjny – pozwala wykonać jedno zapytanie SQL łączące dane z S3, Postgresa i Kafki.
- **DuckDB:** Niezwykle szybki silnik wektorowy *in-process* (odpowiednik SQLite dla analityki kolumnowej OLAP).

3. Rewolucja deklaratywna: dbt (Data Build Tool)

- Zamiast pisać tysiące linii w Pythonie/Javie, inżynier definiuje transformacje w **czystym SQL** połączonym z szablonami Jinja. dbt automatycznie buduje graf zależności (DAG) i wykonuje kod wewnątrz bazy.

23. Warstwa 4: Serwowanie, analityka OLAP i Governance

Jak dostarczyć wartość odbiorcom końcowym i utrzymać ład w danych?

1. Wyspecjalizowane analityczne bazy serwujące (Real-Time OLAP)

Gdy raport w PowerBI musi odpowiedzieć w 10 milisekund przy 10 000 równoległych zapytań:

- **ClickHouse:** Niezwykle szybka kolumnowa baza OLAP o zawrotnej kompresji.
- **Apache Pinot / Apache StarRocks:** Architektury serwowania analitycznego w czasie rzeczywistym.

2. Warstwa Zarządzania i Ładu (Data Governance & Catalog)

Bez metadanych każda platforma powróci do stanu bagna danych:

- **Katalogi danych: Unity Catalog** (Databricks), **Apache Atlas**, **OpenMetadata**, **AWS Glue Catalog**.
- **Kontrakty danych (Data Contracts):** Formalne umowy API pomiędzy producentami danych (zespoły software) a konsumentami (zespoły analityczne) wymuszające schemat przed zapisem.

24. Taksonomia rynkowa: Jak nie utonąć w gąszczu narzędzi?

Warstwa	Open Source	AWS (Managed)	Azure (Managed)	GCP (Managed)	SaaS / Niezależne
1. Ingestion	Debezium, Airbyte, Kafka	AWS DMS, Kinesis, MSK	Event Hubs, Data Factory	Cloud Pub/Sub, Datastream	Fivetran
2. Storage / Formats	S3 API, Delta Lake, Iceberg	S3, AWS Glue Tables	ADLS Gen2, OneLake	GCS, BigLake	Tabular, MinIO
3. Compute	Apache Spark, Trino, dbt, Flink	EMR, Athena	Synapse Spark, Fabric	Dataproc, BigQuery	Databricks, Snowflake
4. Serving (OLAP)	ClickHouse, StarRocks, Pinot	Amazon Redshift	Azure Synapse SQL	BigQuery BI Engine	ClickHouse Cloud
Governance	OpenMetadata, Amundsen	AWS Lake Formation	Microsoft Purview	Dataplex	Unity Catalog

Reguła wyboru technologicznego:
Firmy rzadko budują stos oparty na rozwiązaniach tylko jednego dostawcy. Najczęstszy model enterprise to **hybrydowy Lakehouse**: Cloud Storage + Otwarte Formaty Tabelaryczne (Delta/Iceberg) + Elastyczny Compute (Databricks/Snowflake).

25. Złota macierz kompromisów architektonicznych

Nie istnieje architektura "idealna". Każdy wybór inżynierski to kompromis (*Trade-off*):

Kryterium	Architektura Lambda	Architektura Kappa	Architektura Medallion (Lakehouse)
Latencja danych	Mieszana (sekundy w Speed, godziny w Batch)	Bardzo niska (milisekundy / sekundy)	Konfigurowalna (od sekund w streamingu do minut/godzin w batchu)
Złożoność kodu	**Bardzo wysoka (Dual Codebase)**	Niska (jedno API strumieniowe)	Średnia/Niska (unifikacja SQL/DataFrames)
Koszt przechowywania	Średni	**Wysoki (długa retencja w Kafce)**	**Bardzo niski (Object Storage S3)**
Złożone złączenia analityczne	Łatwe w warstwie wsadowej	**Piekielnie trudne w locie**	**Bardzo łatwe i zoptymalizowane**
Odzyskiwanie po awarii	Złożone (synchronizacja warstw)	Proste (reprocessing logu)	Proste (mechanizm Time Travel w ACID)

26. Studia decyzyjne dla inżynierów (Dyskusja na sali)

Scenariusz A: Wykrywanie nadużyć w transakcjach bankowych

Bank procesuje 5 000 płatności kartą na sekundę. Jeśli transakcja wykazuje cechy kradzieży, autoryzacja musi zostać odrzucona w czasie poniżej 150 ms. Dodatkowo raz w miesiącu analitycy ryzyka trenują nowe modele na pełnej historii 5 lat.

👉 **Pytanie:** Jaką architekturę wybierzesz dla części autoryzacyjnej, a jaką dla analityczno-treningowej?

Scenariusz B: Raportowanie sprzedaży dla platformy E-Commerce

Sklep internetowy sprzedaje towary w 20 krajach. Posiada dane z 40 systemów SaaS (reklamy, magazyny, kurierzy, płatności). Zarząd potrzebuje odświeżania raportów 2 razy na dobę, a analitycy stale zmieniają definicje marży brutto.

👉 **Pytanie:** Czy zaproponujesz architekturę Kappa ze strumieniowaniem zdarzeń, czy ELT w architekturze Medallion? Dlaczego?

27. Złote zasady architekta Big Data

1. **Nigdy nie niszcz surowych danych źródłowych:**
   - Warstwa Bronze to Twoja polisa ubezpieczeniowa. Jeśli kod transformacji zawiedzie, zawsze możesz przeliczyć historię na nowo.
2. **Nie płać za streaming tam, gdzie wystarczy mikro-wsad:**
   - 90% potrzeb biznesowych nie wymaga latencji 10 milisekund. Jeśli biznes czyta raport raz na godzinę, przetwarzanie w strumieniu ciągłym to marnowanie budżetu.
3. **Standaryzuj na otwartych formatach, nie na silnikach:**
   - Silniki obliczeniowe przemijają (MapReduce -> Spark -> DuckDB -> ?), ale dane składowane w otwartych formatach tabelarycznych (Delta Lake / Iceberg) przetrwają dekady.
4. **Brak metadanych to gwarancja porażki:**
   - Jeśli nie wiesz, kto stworzył tabelę, jak często jest odświeżana i jakie są typy jej kolumn – masz bagno danych, bez względu na to, jak nowoczesne technologie kupiłeś.

Podsumowanie Wykładu 2

- **Ewolucja paradygmatów:** Tani chmurowy storage doprowadził do triumfu **ELT** (*Schema-on-Read*) nad tradycyjnym **ETL** (*Schema-on-Write*).
- **Lekcja z historii: Architektura Lambda** cierpiała na problem podwójnego kodu (*Dual Codebase*), a **Architektura Kappa** napotkała bariery kosztu retencji i złożonych złączeń historycznych.
- **Problem produkcyjny:** Bezkrytyczne wrzucanie plików do S3 bez schematu i transakcji zamienia Data Lake w **Data Swamp** i generuje silosy analityczne.
- **Wzorzec Lakehouse: Architektura Medallion (Bronze ➔ Silver ➔ Gold)** z otwartymi formatami tabelarycznymi zapewnia porządek, jakość i transakcyjność ACID.
- **Taksonomia:** Nowoczesna platforma dzieli się na 4 niezależne warstwy: **Ingestion**, **Storage**, **Compute**, **Serving & Governance**.

Dziękuję za uwagę!

Następny wykład (W03):

Przechowywanie danych: Formaty, Data Lake i Lakehouse

Anatomia zapytań analitycznych • Formaty wierszowe vs kolumnowe (Parquet, ORC) • ACID na plikach: Delta Lake & Apache Iceberg

Specyfikacje, materiały źródłowe i diagramy dostępne w repozytorium kursu.